

Team

3: Dealer + Retailer

DMS Team 3

July 2026

dealer.c retailer.c — Build Guide

Team 3 — Dealer + Retailer

You build **dealer.c**, **dealer.h**, **retailer.c**, and **retailer.h**. This is the **ordering logic**: stock moves from company → dealer → retailer. hardest part of DMS.

Your deliverable: login by ID, view stock, place orders with validation, transfer stock between levels.

Team	Files	Role
Team 1	data.c, data.h, display.c, display.h	Arrays + UI (YOU depend on this)
Team 2	main.c, company.c, company.h	Calls your dealer_menu() and retailer_menu()
Team 3 (YOU)	dealer.c, dealer.h, retailer.c, retailer.h	Dealer + Retailer panels + orders

Three-team split

1. What You Are Building

1.1 dealer.c

- Dealer login by ID (search dealer_id array).
- View own stock, view company products.
- **order_from_company()** — validate and transfer stock.

1.2 retailer.c

- Retailer login by ID; show linked dealer.
- View own stock, view dealer products.
- **order_from_dealer()** — same validation pattern, different arrays.

2. What You Need From Teammates

From Team 1	Use for
company_stock[], dealer_stock[], retailer_stock[]	Order transfers
product_id[], product_name[], product_total	Search products
dealer_id[], dealer_name[], dealer_total	Dealer login
retailer_id[], retailer_dealer[]	Retailer login + link
show_company_products()	Before dealer order
show_dealer_products(i)	Before retailer order
verify_password(), print_order_summary()	Login + success screen

Team 1 dependencies

Team 2's main.c will call **dealer_menu()** and **retailer_menu()** — deliver dealer.h and retailer.h early so they can compile.

3. Build Order

1. dealer.h / retailer.h — one function each.
2. dealer.c — show_dealer_stock(), then order_from_company(), then dealer_menu().
3. Test dealer alone with mini main.
4. retailer.c — show_retailer_stock(), order_from_dealer(), retailer_menu().
5. Full integration with Team 2's main.c.

4. Step-by-Step: dealer.c

Step 4.1 — show_dealer_stock(dealer_index)

c

```
void show_dealer_stock(int dealer_index) {
    int i;
    print_subtitle("My Stock");
    printf("Logged in as: %s\n", dealer_name[dealer_index]);
    print_table_product_header();
    for (i = 0; i < product_total; i++) {
        print_table_product_row(product_id[i], product_name[i],
                                dealer_stock[dealer_index][i]);
    }
    print_table_footer();
}
```

Step 4.2 — order_from_company() validation chain

1. Call show_company_products().
2. Read product_id_input and quantity.
3. Search product — if not found, error + return.
4. If quantity <= 0, error + return.
5. If company_stock < quantity, error + return.
6. Transfer: company -= qty, dealer += qty.
7. print_order_summary(...).

c

```
company_stock[product_index] -= quantity;
dealer_stock[dealer_index][product_index] += quantity;
print_order_summary(product_name[product_index], quantity,
    "Company", dealer_name[dealer_index]);
```

Step 4.3 — dealer_menu() login loop

Show dealer table → scanf dealer ID → for-loop search with found flag → verify_password
→ do-while menu calling options 1–3.

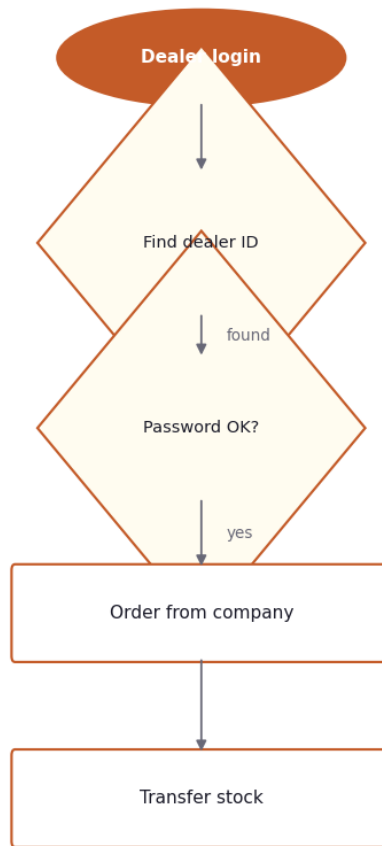


Figure 1: Dealer order path

5. Step-by-Step: retailer.c

Step 5.1 — Find linked dealer

C

```
dealer_index = 0;
for (i = 0; i < dealer_total; i++) {
    if (dealer_id[i] == retailer_dealer[retailer_index]) {
        dealer_index = i;
        break;
    }
}
```

Step 5.2 — order_from_dealer()

Same pattern as dealer order, but:

- Show dealer with show_dealer_products(dealer_index).
- Check dealer_stock[dealer_index][product_index] >= quantity.
- dealer_stock -= qty; retailer_stock += qty.

C

```
dealer_stock[dealer_index][product_index] -= quantity;
retailer_stock[retailer_index][product_index] += quantity;
print_order_summary(product_name[product_index], quantity,
    dealer_name[dealer_index], retailer_name[retailer_index]);
```

Step 5.3 — retailer_menu() login table

Print custom table with ID, Retailer Name, Linked To using print_table_retailer_row(). Then ID search, find dealer_index, password, menu loop.

6. The found Pattern (Copy Everywhere)

C

```
found = 0;
product_index = 0;
for (i = 0; i < product_total; i++) {
    if (product_id[i] == product_id_input) {
        found = 1;
        product_index = i;
        break;
    }
}
if (found == 0) {
    print_error("Product not found.");
    return;
}
```

7. Test Your Part

Mini main for dealer only

BASH

```
gcc test_dealer.c data.c display.c dealer.c -o test3
./test3
```

C

```
/* test_dealer.c */
#include "data.h"
#include "dealer.h"
int main() { init_data(); dealer_menu(); return 0; }
```

Full integration

BASH

```
gcc main.c data.c display.c company.c dealer.c retailer.c -o dms
./dms
```

Test	Input	Expected stock change
Dealer order	ID 1, Rice qty 10	Company Rice 100→90, Dealer 20→30
Over-order	Rice qty 5000	Error, no change
Retailer order	ID 1, Rice qty 5	Dealer Rice 30→25, Retailer 5→10
Bad product ID	ID 99	Product not found

Critical tests

Location	After dealer+retailer tests
Company Rice	90
Dhaka Dealer Rice	25
Mirpur Shop Rice	10

Verify with Company panel (Team 2) option 3

8. Team Workflow Timeline

Day	Team 1	Team 2	Team 3
1	data.h + data.c arrays	company.h stub	dealer.h + retailer.h stubs
2	display.c complete	company.c views	dealer stock + order
3	Handoff + test	main.c + stubs	retailer.c complete
4	Integration test all	Remove stubs	End-to-end orders

Suggested 4-day plan

Team 3 checklist

- dealer_menu() and retailer_menu() compile and link from main.c.
- All three validation gates before every transfer.
- Stock never goes negative on valid paths.
- Commit: dealer.c, dealer.h, retailer.c, retailer.h.